FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
Pós Graduação Inteligência Artificial Para Devs

TECH CHALLENGE FASE 1
INTELIGÊNCIA ARTIFICIAL NA PREDIÇÃO DE OSTEOPOROSE:
Abordagem Multimodal para Diagnóstico Auxiliar

Brasil

2026

Gabriel Lopes Guimarães - RM378344
Lucas Muniz Vieira Souto - RM378518
Marcos Pinheiro Moura - RM377337
Maria Luiza Assumpção Tozi - RM378682
Saulo Roberto de Sena - RM378146

INTELIGÊNCIA ARTIFICIAL NA PREDIÇÃO DE OSTEOPOROSE:
Abordagem Multimodal para Diagnóstico Auxiliar

Trabalho apresentado como requisito para obtenção de nota na primeira fase do Tech Challenge, desenvolvido no âmbito da Pós-Tech em Inteligência Artificial para Desenvolvedores da Faculdade de Informática e Administração Paulista (FIAP).

Coordenador: Professor José Neto

Brasil

2026

LISTA DE IMAGENS

Imagem 1 – Representação do resultado do tratamento de dados.
Imagem 2 – Importância dos dados para o treinamento
Imagem 3 - Preparação das entradas do modelo
Imagem 4 - Treinamento e busca de hiperparâmetros do modelo
Imagem 5 - Matrizes de confusão da regressão logística (validação e teste)
Imagem 6 - Configuração e treinamento do modelo
Imagem 7 - Validação cruzada estratificada do modelo
Imagem 8 - Matriz de confusão (Random Forest)
Imagem 9 - Gráfico com SHAP
Imagem 10 - Configuração e busca de hiperparâmetros do modelo
Imagem 11 - Matriz de confusão do modelo
Imagem 12 - Curva ROC
Imagem 13 - Importância das variáveis do modelo
Imagem 14 – Gráfico relacionado à importância por permutação
Imagem 15 – Comparação entre resultados de modelos
Imagem 16 - Modelo Escolhido
Imagem 17 - Padronização para criação de novas imagens
Imagem 18 - Saída do algoritmo de criação das imagens
Imagem 19 - Distribuição entre Treino, Validação e Teste
Imagem 20 - Impressão do Primeiro Lote de Treinamento
Imagem 21 - Exemplo Treinamento MobileNet
Imagem 22 - Treino MobileNet 50 épocas
Imagem 23 - Treino MobileNet 20 épocas
Imagem 24 - Matriz de Confusão MobileNet 20 épocas
Imagem 25 - Treino RestNet50 50 épocas
Imagem 26 - Treino RestNet50 34 épocas
Imagem 27 - Matriz de Confusão RestNet50 34 épocas
Imagem 28 - Treino EfficientNetB0 50 épocas
Imagem 29 - Treino EfficientNetB0 36 épocas
Imagem 30 - Matriz de Confusão EfficientNetB0 36 épocas
Imagem 31 - Comparação entre Modelos desafio
Imagem 32 - Escolha do melhor modelo desafio

LISTA DE TABELAS

Tabela 1 – Mapeamento da conversão de variáveis binárias.
Tabela 2 – Renomeação de variáveis e correspondência semântica.
Tabela 3 – Conversão de variáveis categóricas em variáveis indicadoras (dummies).
Tabela 4 – Imputação e codificação para tratamento de valores ausentes.
Tabela 5 – Parâmetros e configurações padronizadas adotadas no treinamento dos modelos.

SUMÁRIO

1 INTRODUÇÃO

2 BASE DE DADOS

2.1 Descrição

2.2 Tratamento

2.2.1 Remoção De Identificadores E Codificação Binária

2.2.2 Renomeação Das Colunas

2.2.3 Conversão De Variáveis Categóricas

2.2.4 Tratamento Dos Valores Ausentes

2.2.5 Resultado Do Tratamento

3 DIVISÃO TREINO, VALIDAÇÃO E TESTE

4 TREINAMENTOS

4.1 Regressão Logística

4.2 Random Forest

4.3 Gradient Boosting

5 RESULTADOS E ESCOLHA DO MELHOR MODELO

5.1 Modelo Escolhido

6 DESAFIO

6.1 Base De Dados

6.2 Manipulação

6.3 Divisão Treino, Validação E Teste

6.4 Treinamentos

6.4.1 Mobilenet

6.4.2 Resnet50

6.4.3 Efficientnetb0

6.5 Comparando Resultados

6.6 Modelo Escolhido

7 PERGUNTA FINAL

7.1 Informações Complementares

7.1.1 Repositório

7.1.3 Apresentação

8 REFERÊNCIAS

1 INTRODUÇÃO

A osteoporose é uma doença metabólica óssea caracterizada pela perda progressiva de massa óssea e pela deterioração da microarquitetura do osso, o que torna o esqueleto mais frágil e suscetível a fraturas mesmo diante de traumas leves. Por evoluir de forma silenciosa, sem sintomas evidentes até a ocorrência da primeira fratura, a doença frequentemente é diagnosticada de maneira tardia, quando suas complicações já comprometem a mobilidade, a autonomia e a qualidade de vida do indivíduo (PEDRO; PLAPLER; SZEJNFELD, 2021).

A relevância do tema justifica-se por seu impacto crescente sobre a saúde pública. Estima-se que a osteoporose afete mais de 200 milhões de pessoas no mundo e cerca de 10 milhões no Brasil, atingindo sobretudo mulheres na pós-menopausa e a população idosa. As fraturas dela decorrentes, especialmente as de quadril, associam-se a elevadas taxas de morbidade e mortalidade, com projeções de que os casos cheguem a 160 mil por ano até 2050 no país (PEDRO; PLAPLER; SZEJNFELD, 2021).

Diante desse cenário, e considerando que o diagnóstico precoce e as medidas preventivas podem mudar a vida de muitos pacientes, este trabalho tem como objetivo empregar técnicas de machine learning e deep learning no treinamento de modelos computacionais para a criação de scripts voltados ao auxílio da equipe médica no diagnóstico, na identificação precoce e no suporte à tomada de decisão clínica.

2 BASE DE DADOS

Fonte dos dados utilizados para o treinamento de machine learning foi extraído da plataforma Kaggle[1], conjunto “Lifestyle Factors Influencing Osteoporosis”, disponibilizado por Amit Kulkarni. A base reúne características demográficas, histórico pessoal e familiar, hábitos e condições de saúde associados à osteoporose. Esses registros são utilizados para desenvolver e comparar métodos de classificação da presença ou ausência da doença.

[1] Fonte Disponível em: https://www.kaggle.com/datasets/amitvkulkarni/lifestyle-factors-influencing-osteoporosis

2.1 Descrição

O conjunto original contém 1.958 registros e 16 colunas, incluindo o identificador Id e a variável de resposta Osteoporosis. As demais colunas descrevem idade, gênero, alterações hormonais, histórico familiar, raça/etnia, peso corporal, ingestão de cálcio e vitamina D, atividade física, tabagismo, consumo de álcool, condições médicas, medicamentos e fraturas anteriores.

A variável Osteoporosis já está codificada em duas classes: 0, ausência de osteoporose, e 1, presença de osteoporose. A distribuição original é equilibrada, com 979 registros em cada classe. Id é apenas uma identificação dos registros e não representa um fator clínico de previsão.

2.2 Tratamento

Tratar os dados antes de treinar um modelo de Machine Learning é indispensável para eliminar inconsistências, ruídos e disparidades de escala que prejudicam a aprendizagem do algoritmo. Ao transformar e reformatar as variáveis, garantimos que a inteligência artificial identifique padrões reais em vez de ruídos, permitindo uma convergência mais rápida e previsões precisas. A seguir, detalharemos cada etapa do processo de tratamento dos dados e seu impacto no modelo.

2.2.1 Conversão de Variáveis Binárias

A etapa de pré-processamento dos dados, realizou-se a codificação das variáveis categóricas dicotômicas em formato numérico. Esta conversão para o formato binário (0 ou 1) é um passo fundamental para permitir a interpretação adequada dos dados pelos algoritmos de machine learning ,garantindo a padronização das entradas do modelo.A Tabela 1 detalha o mapeamento efetuado entre as variáveis clínicas e comportamentais analisadas, o nome das colunas correspondentes na base de dados e a atribuição dos respetivos valores numéricos:

Tabela 1 – Mapeamento da conversão de variáveis binárias.

Variável	Coluna	Valor = 1	Valor = 0
Gênero	Gender	Feminino	Masculino
Alterações hormonais	Hormonal Changes	Pós-menopausa	Normal

Variável	Coluna	Valor = 1	Valor = 0
Histórico familiar	Family History	Sim	Não
Peso corporal	Body Weight	Abaixo do peso	Normal
Ingestão de cálcio	Calcium Intake	Baixa	Adequada
Ingestão de vitamina D	Vitamin D Intake	Insuficiente	Suficiente
Atividade física	Physical Activity	Ativo	Sedentário
Tabagismo	Smoking	Sim	Não
Fraturas anteriores	Prior Fracture	Sim	Não

2.2.2 Renomeação das Colunas

Renomeamos algumas colunas para deixar claro o significado do valor 1. A alteração muda apenas os nomes das variáveis, sem afetar os valores contidos nelas.

Tabela 2 – Renomeação de variáveis e correspondência semântica.

Nome original	Nome após renomeação	Significado do valor 1
Body Weight	Underweight	Abaixo do peso
Calcium Intake	Low Calcium Intake	Baixa ingestão de cálcio
Vitamin D Intake	Low Vitamin D Intake	Ingestão insuficiente de vitamina D
Medications	Corticosteroid Use	Uso de corticoides

2.2.3 Conversão de Variáveis Categóricas

As variáveis raça/etnia (Race/Ethnicity) e condições médicas (Medical Conditions), que possuem mais de duas categorias, foram convertidas em colunas indicadoras por meio de pd.get_dummies.

Tabela 3 – Conversão de variáveis categóricas em variáveis indicadoras (dummies).

Variável	Categoria
Raça/etnia	Afro-americana (African American)
Raça/etnia	Asiática (Asian)
Raça/etnia	Caucasiana (Caucasian)
Condições médicas	Hipertireoidismo (Hyperthyroidism)
Condições médicas	Artrite reumatoide (Rheumatoid Arthritis)

Cada coluna indica com 1 a presença da categoria e com 0 sua ausência, sem estabelecer uma ordem numérica entre as categorias.

2.2.4 Tratamento dos Valores Ausentes

Nas variáveis “Alcohol Consumption” e “Medications”, foram adotadas as seguintes codificações:

Tabela 4 – Imputação e codificação para tratamento de valores ausentes.

Variável	Valor = 1	Valor = 0
Consumo moderado de álcool	Sim	Sem registro
Uso de corticoides	Sim	Sem registro

Essa decisão interpreta a ausência de registro como ausência do consumo ou medicamento indicado. Trata-se de uma premissa adotada no tratamento dos dados, e não de uma confirmação clínica individual.

2.2.5 Resultado do Tratamento

Ao final dessas transformações, as variáveis explicativas ficaram em formato numérico e a coluna Osteoporosis foi mantida como variável-alvo.

A preparação dos dados é comum nos métodos de classificação. Eventuais transformações adicionais necessárias a cada algoritmo são descritas nas respectivas etapas de treinamento.

```
display(df)
```

	Age	Gender	Hormonal Changes	Family History	Underweight	Low Calcium Intake	Low Vitamin D Intake	Physical Activity	Smoking	Alcohol Consumption	Corticosteroid Use	Prior Fractures	Osteoporosis	African American	Asian	Caucasian	Hyperthyroidism	Rheumatoid Arthritis
0	69	1	0	1	1	1	0	0	1	1	1	1	1	0	1	0	0	1
1	32	1	0	1	1	1	0	0	0	0	0	1	1	0	1	0	0	0
2	89	1	1	0	0	0	0	1	0	1	1	0	1	0	0	1	1	0
3	78	1	0	0	1	0	1	0	1	0	1	0	1	0	0	1	0	1
4	38	0	1	1	0	1	0	1	1	0	0	1	1	1	0	0	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1953	19	1	0	1	0	0	0	0	1	1	0	1	0	1	0	0	0	1
1954	23	1	1	1	1	1	1	1	0	0	1	0	0	0	0	1	0	0
1955	34	1	1	0	1	1	0	0	0	0	0	0	0	1	0	0	1	0
1956	25	0	1	0	0	1	1	0	1	0	1	1	0	1	0	0	0	1
1957	26	1	1	0	1	0	0	0	1	0	1	0	0	1	0	0	0	1

1958 rows × 18 columns

Imagem 1 – Representação do resultado do tratamento de dados.

2.2.6 Especificação dos Dados

Após o tratamento dos dados, foram criados gráficos para uma análise exploratória dedicada à distribuição das variáveis e ao seu impacto nos treinos que seriam feitos mais adiante. A verificação dos gráficos relativos à quantificação dos atributos nos grupos com e sem osteoporose revela um comportamento bastante

homogêneo e simétrico no conjunto de dados. Características clínicas e hábitos de vida, tais como o uso de corticoides, a atividade física e o baixo consumo de cálcio, mantêm proporções aproximadas aos 50% de presença e ausência em ambas as populações, o que demonstra uma distribuição equilibrada dos dados.

Em relação à importância dos atributos para a aprendizagem do modelo, a avaliação realizada através do algoritmo Random Forest evidencia a influência determinante do fator idade. Quando a variável idade está incluída na modelação, esta assume uma importância elevada em relação aos outros dados, concentrando 63,7% da relevância no processo de decisão, o que reduz individualmente todas as outras características clínicas e comportamentais para valores abaixo dos 3%.

Por outro lado, ao analisar o cenário com a remoção da variável idade, observa-se uma redistribuição da importância entre os restantes atributos. Fatores como o baixo consumo de cálcio (8,3%), o género (7,8%), o consumo de álcool (7,7%) e o baixo consumo de vitamina D (7,6%) passam a desempenhar papéis de relevo na tomada de decisão. Esta comparação torna-se essencial para compreender que, embora a idade seja o fator isolado mais determinante no diagnóstico, o modelo é igualmente capaz de captar a importância dos fatores de estilo de vida e marcadores clínicos quando o fator idade é retirado da análise.

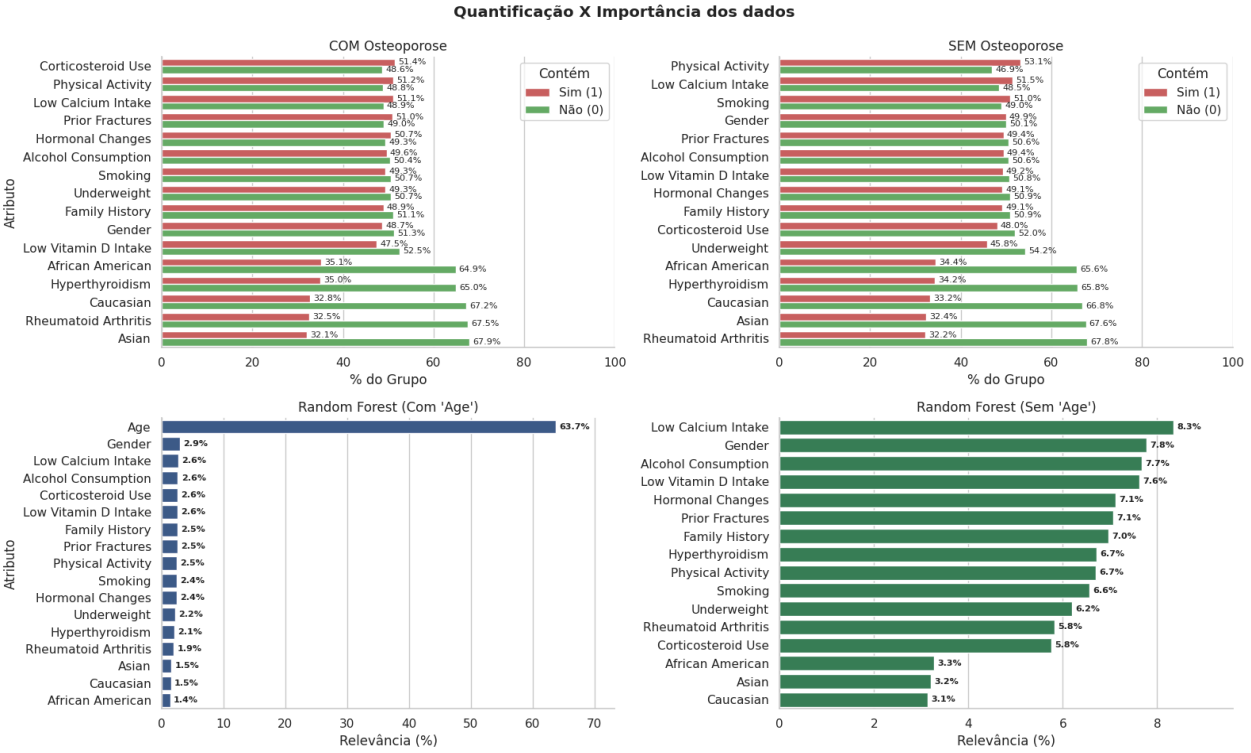


Imagem 2 – Importância dos dados para o treinamento

3 DIVISÃO TREINO, VALIDAÇÃO E TESTE

Após a preparação dos dados que envolveu a identificação das variáveis preditoras e da variável-alvo, além da verificação da estrutura do conjunto de dados e da adequação das informações para o treinamento dos modelos. A variável Age foi mantida em seu formato numérico original, preservando os valores de idade disponíveis na base.

Para avaliar o desempenho dos modelos, os dados foram divididos em conjuntos de treinamento, validação e teste. A divisão foi realizada de forma estratificada em relação à variável-alvo, buscando preservar a proporção das classes nos subconjuntos. O conjunto de treinamento foi utilizado para o ajuste dos modelos, enquanto os conjuntos de validação e teste serviram para apoiar a avaliação. Como o tamanho do dataset é relativamente reduzido e a idade é um fator determinante, os conjuntos de validação e teste foram utilizados após o treinamento para comparar a variação nos resultados obtidos em cada execução.

4 TREINAMENTOS

O treinamento teve como objetivo desenvolver e avaliar modelos de aprendizado de máquina capazes de identificar a ocorrência de osteoporose a partir das características disponíveis no conjunto de dados.

Para permitir uma avaliação mais consistente, foi utilizada a validação cruzada estratificada *Stratified K-Fold Cross-Validation* com cinco partições, conforme a configuração adotada para os modelos correspondentes. Esse procedimento permite avaliar o desempenho em diferentes subdivisões dos dados de treinamento, mantendo aproximadamente a proporção das classes em cada partição.

A seleção e o ajuste dos hiperparâmetros foram realizados de acordo com a abordagem definida para cada algoritmo. No caso do Random Forest, foi utilizada a busca aleatória de combinações de hiperparâmetros (*RandomizedSearchCV*), com o objetivo de identificar uma configuração adequada segundo a métrica de avaliação selecionada.

O desempenho dos modelos foi analisado por meio de métricas complementares, incluindo acurácia (*Accuracy*), precisão (*Precision*), revocação (*Recall*), F1-score e área sob a curva ROC (*ROC-AUC*). A análise conjunta dessas

métricas permite observar diferentes aspectos do desempenho, evitando que a comparação seja baseada exclusivamente na proporção de classificações corretas.

Após o treinamento e a seleção das configurações, o desempenho final deve ser avaliado no conjunto de teste reservado para essa finalidade, sem utilizá-lo no processo de ajuste dos hiperparâmetros.

4.1 Regressão logística

A Regressão Logística foi implementada com LogisticRegression, utilizando solver="liblinear", max_iter=5000 e random_state=42. O modelo recebeu 16 variáveis explicativas após a remoção de Caucasian, adotada como categoria de referência. A padronização foi aplicada somente à variável Age, por meio de um ColumnTransformer integrado a um Pipeline, garantindo que os parâmetros de transformação fossem ajustados apenas com os dados de treinamento e evitando vazamento de informações.

```
X_train_lr = X_train
X_val_lr = X_val
X_test_lr = X_test

X_train_lr = X_train_lr.drop(columns=['Caucasian'])
X_val_lr = X_val_lr.drop(columns=['Caucasian'])
X_test_lr = X_test_lr.drop(columns=['Caucasian'])
preprocessor = ColumnTransformer(
    transformers=[
          ('age', StandardScaler(), ['Age'])
      ],
     remainder='passthrough'
)
```

Imagem 3 - Preparação das entradas do modelo

A otimização dos hiperparâmetros foi realizada com GridSearchCV, avaliando 14 combinações em cinco partições estratificadas (StratifiedKFold). A melhor configuração apresentou C=0.1 e penalização L1, alcançando F1-score médio de 0,8480 na validação cruzada. Na validação separada, o modelo obteve acurácia de 0,8503, precisão de 0,9120, recall de 0,7755, F1-score de 0,8382 e ROC-AUC de 0,9127. No conjunto de teste, os resultados foram de 0,8027 para acurácia, 0,8938 para precisão, 0,6871 para recall, 0,7769 para F1-score e 0,8640 para ROC-AUC, indicando uma redução moderada no desempenho em dados não utilizados durante o treinamento e a seleção dos hiperparâmetros.

```python
# Modelo e ajuste de hiperparâmetros
logistic_regression = LogisticRegression(solver='liblinear', max_iter=5000, random_state=42)

pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', logistic_regression)])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

param_grid = {'classifier__C': [0.001, 0.01, 0.1, 1, 10, 100, 1000], 'classifier__penalty': ['l1', 'l2']}
grid_search = GridSearchCV( pipeline, param_grid, scoring='f1', cv=cv, n_jobs=-1)
grid_search.fit(X_train_lr, y_train)

best_model = grid_search.best_estimator_

print('Melhores parâmetros:', grid_search.best_params_)
print(f'Melhor F1 na validação cruzada: {grid_search.best_score_:.4f}')
```

Imagem 4 - Treinamento e busca de hiperparâmetros do modelo

A análise das matrizes de confusão demonstrou que, na validação, o modelo identificou corretamente 114 casos de osteoporose e 136 casos sem a condição, apresentando 11 falsos positivos e 33 falsos negativos. No teste, foram registrados 101 verdadeiros positivos, 135 verdadeiros negativos, 12 falsos positivos e 46 falsos negativos. Esses resultados evidenciam uma precisão elevada, porém com redução da sensibilidade no conjunto de teste, indicando maior dificuldade na identificação de todos os casos positivos de osteoporose.

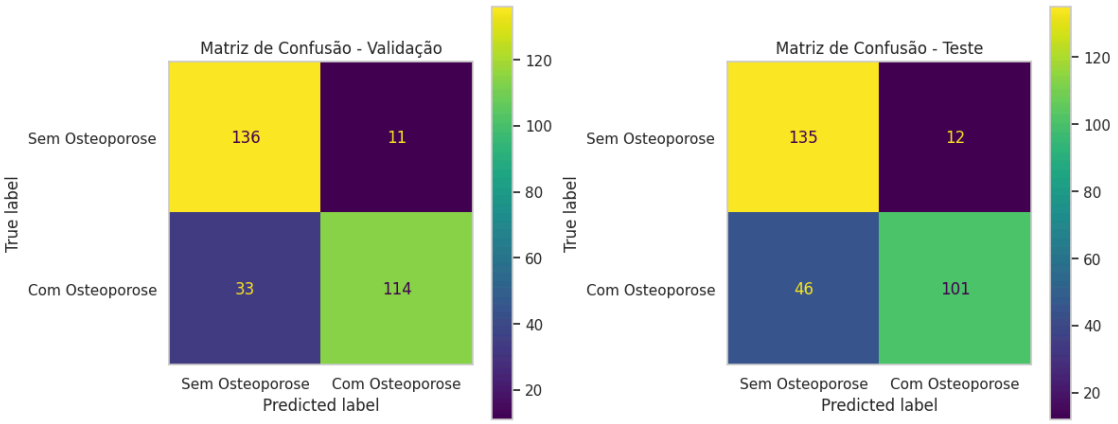


Imagem 5 - Matrizes de confusão da regressão logística (validação e teste)

As curvas ROC apresentaram áreas sob a curva (AUC) de 0,9127 na validação e 0,8640 no teste, demonstrando boa capacidade de discriminação entre as classes. Apesar da redução observada no conjunto de teste, os resultados indicam que o modelo manteve capacidade preditiva relevante em dados não utilizados no treinamento.

4.2 Random Forest

O Random Forest foi utilizado para classificar a presença ou ausência de osteoporose com base em características demográficas, hábitos de vida e histórico de saúde. O modelo foi configurado com 100 árvores de decisão, semente aleatória 42 e class_weight='balanced'.

```python
# Validação cruzada é feita somente dentro do conjunto de treino.
cv_rf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

parametros_rf = {
    "n_estimators": [100, 200, 300, 500, 700],
    "max_depth": [None, 8, 12, 16, 20, 30],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", 0.7, 1.0],
    "class_weight": [None, "balanced"]
}

modelo_base_rf = RandomForestClassifier(random_state=42, n_jobs=1)

busca_rf = RandomizedSearchCV(
    estimator=modelo_base_rf,
    param_distributions=parametros_rf,
    n_iter=60,
    scoring="accuracy",
    cv=cv_rf,
    random_state=42,
    n_jobs=-1,
    refit=True,
    verbose=1
)

busca_rf.fit(X_train_rf, y_train_rf)
modelo_rf = busca_rf.best_estimator_
```

Imagem 6 - Configuração e treinamento do modelo

Na validação cruzada estratificada com cinco partições, apresentou acurácia média de 86,79%, precisão de 96,74%, recall de 76,20%, F1-score de 85,18% e ROC-AUC de 91,54%. No conjunto de validação independente, com 392 registros, obteve acurácia de 82,40%, precisão de 97,74%, recall de 66,33%, F1-score de 79,03% e ROC-AUC de 85,80%.

```python
# Métricas de validação cruzada no conjunto de treino do Random Forest.
cv_rf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

resultados_cv_rf = cross_validate(
    modelo_rf,
    X_train_rf,
    y_train_rf,
    cv=cv_rf,
    scoring={
        "accuracy": "accuracy",
        "precision": "precision",
        "recall": "recall",
        "f1": "f1",
        "roc_auc": "roc_auc"
    },
    n_jobs=-1
)
```

Imagem 7 - Validação cruzada estratificada do modelo

Os resultados mostram alta precisão nas classificações positivas, mas uma limitação na identificação dos casos com osteoporose: no conjunto independente, o modelo reconheceu aproximadamente dois terços dos registros positivos. O F1-score foi considerado relevante para comparar os modelos por combinar precisão e recall.

Nos conjuntos de validação e teste, o modelo apresentou perfeita especificidade, identificando corretamente todos os 147 indivíduos saudáveis Verdadeiros Negativos sem gerar nenhum Falso Positivo. Quanto aos casos confirmados de osteoporose, o algoritmo registrou 121 Verdadeiros Positivos, 26 Falsos Negativos na validação e 109 Verdadeiros Positivos 38 Falsos Negativos no teste, mantendo um bom padrão de classificação e generalização.

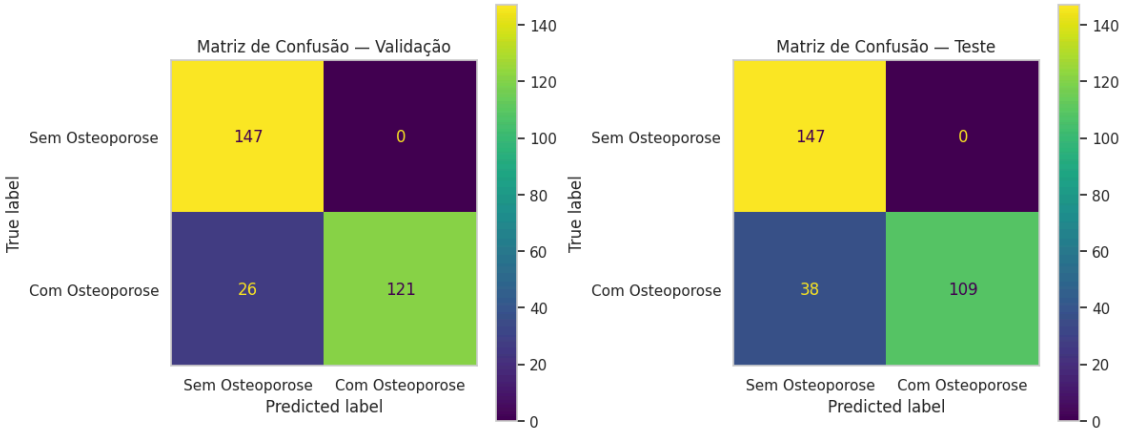


Imagem 8 - Matriz de confusão (Random Forest)

A técnica SHAP, aplicada a até 100 registros de validação, auxiliou na interpretação da influência das variáveis nas previsões.

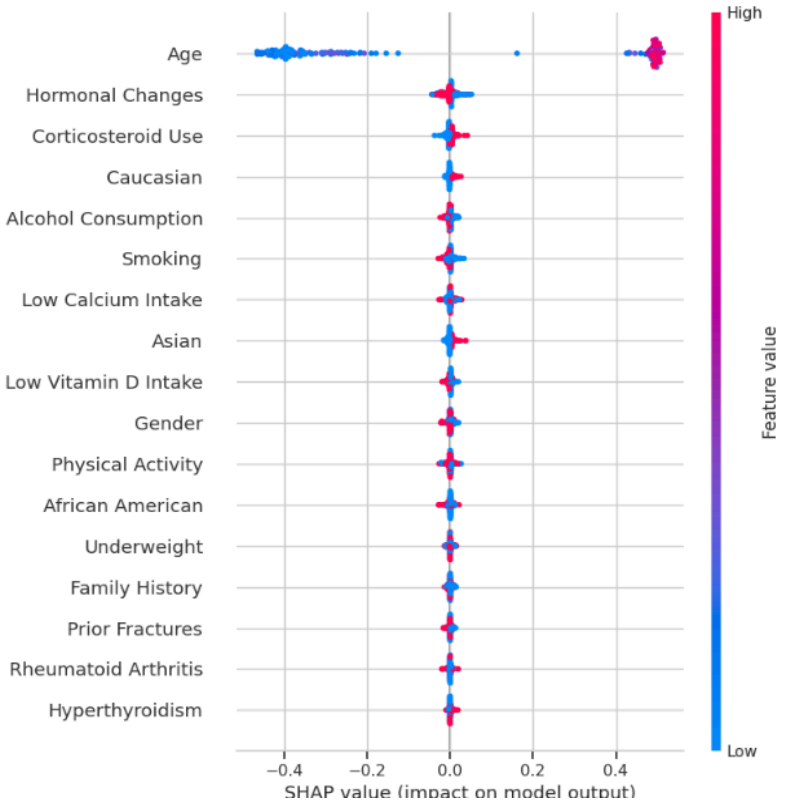


Imagem 9 - Gráfico com SHAP

4.3 Gradient Boosting

O Gradient Boosting foi implementado com XGBClassifier, da biblioteca XGBoost, utilizando 17 variáveis explicativas, sem padronização adicional. Os 1.958 registros foram divididos de forma estratificada em aproximadamente 70% para treinamento (1.370), 15% para validação (294) e 15% para teste (294), com random_state=42, preservando a proporção das classes em cada conjunto.

```python
xgb = XGBClassifier(
    objective='binary:logistic',
    eval_metric='logloss',
    random_state=42,
    n_jobs=1
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

param_grid_xgb = {
    "n_estimators": [100, 200],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.01, 0.05, 0.1],
    "subsample": [0.8, 1.0],
}
grid_search_xgb = GridSearchCV(XGBClassifier(eval_metric="logloss", random_state=42), param_grid_xgb, scoring='f1', cv=3, n_jobs=-1)

#grid_search_xgb = GridSearchCV(xgb, param_grid_xgb, scoring='f1', cv=cv, n_jobs=-1)
grid_search_xgb.fit(X_train, y_train)
```

Imagem 10 - Configuração e busca de hiperparâmetros do modelo

A otimização com GridSearchCV avaliou 36 combinações de hiperparâmetros em três partições, selecionando 200 árvores, profundidade máxima de 7, taxa de aprendizado de 0,01 e subsample=1.0, com F1 médio de 0,9154. A análise de possível overfitting apresentou F1-score de 0,9246 no treinamento e 0,8996 na validação, indicando uma pequena redução de desempenho. Na validação, o modelo obteve acurácia de 0,9082, precisão de 0,9918, recall de 0,8231 e ROC-AUC de 0,9246. No teste, apresentou acurácia de 0,8673, precisão de 0,9909, recall de 0,7415, F1-score de 0,8482 e ROC-AUC de 0,8798.

As matrizes de confusão registraram apenas um falso positivo em cada conjunto, porém 26 falsos negativos na validação e 38 no teste, demonstrando elevada precisão, mas limitações na identificação de todos os casos de osteoporose.

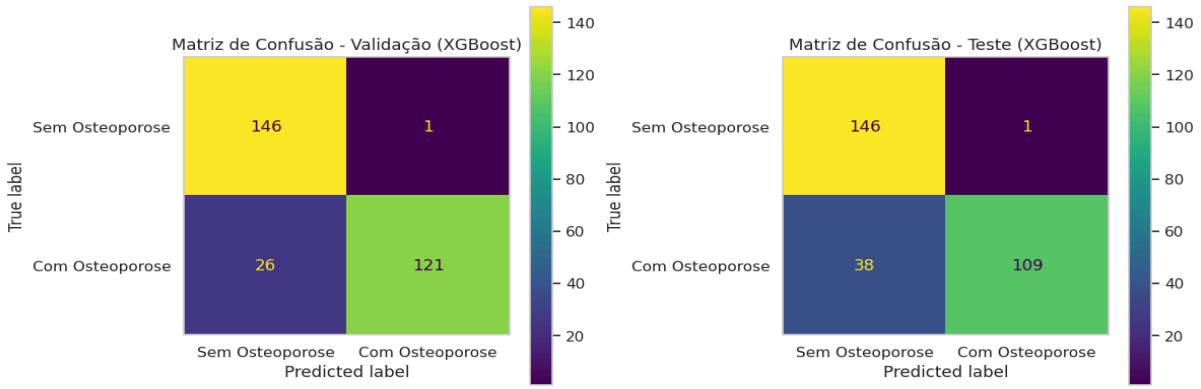


Imagem 11 - Matriz de confusão do modelo

As curvas ROC apresentaram áreas sob a curva (AUC) de 0,9246 na validação e 0,8798 no teste, evidenciando boa capacidade de discriminação entre as classes. Embora tenha ocorrido uma redução no desempenho no conjunto de teste, o modelo manteve resultados relevantes em dados não utilizados no treinamento.

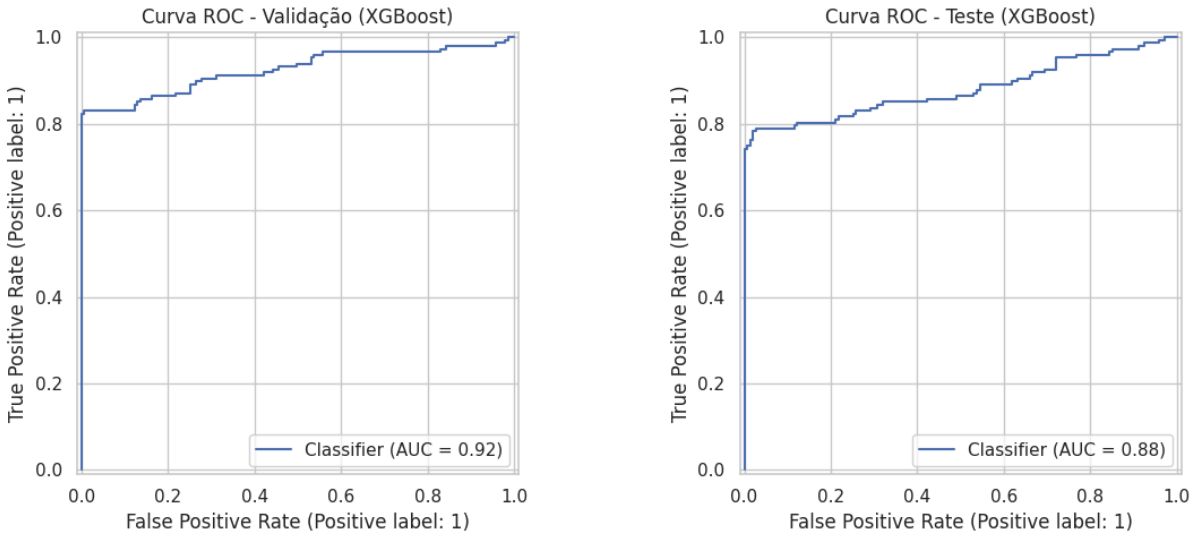


Imagem 12 - Curva ROC

A análise da importância das variáveis destacou a idade (Age) como o principal fator preditivo, com importância por ganho de 0,6316 e redução média de 0,4473 no F1-score ao ser embaralhada em 20 repetições. As demais variáveis apresentaram importâncias médias por permutação próximas de zero, com pequenas contribuições positivas ou negativas. Esses resultados indicam forte dependência do modelo em relação à idade, enquanto as demais características apresentaram contribuição individual limitada para o F1-score nas condições avaliadas.

```
df_importancia_xgb = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance (gain)': best_xgb.feature_importances_
})

perm_xgb = permutation_importance(best_xgb, X_val, y_val, scoring='f1', n_repeats=20, random_state=42, n_jobs=-1)
df_importancia_xgb['Importance (permutation)'] = perm_xgb.importances_mean

df_importancia_xgb = df_importancia_xgb.sort_values('Importance (permutation)', ascending=False)
display(df_importancia_xgb.round(4))
```

Imagem 13 - Importância das variáveis do modelo

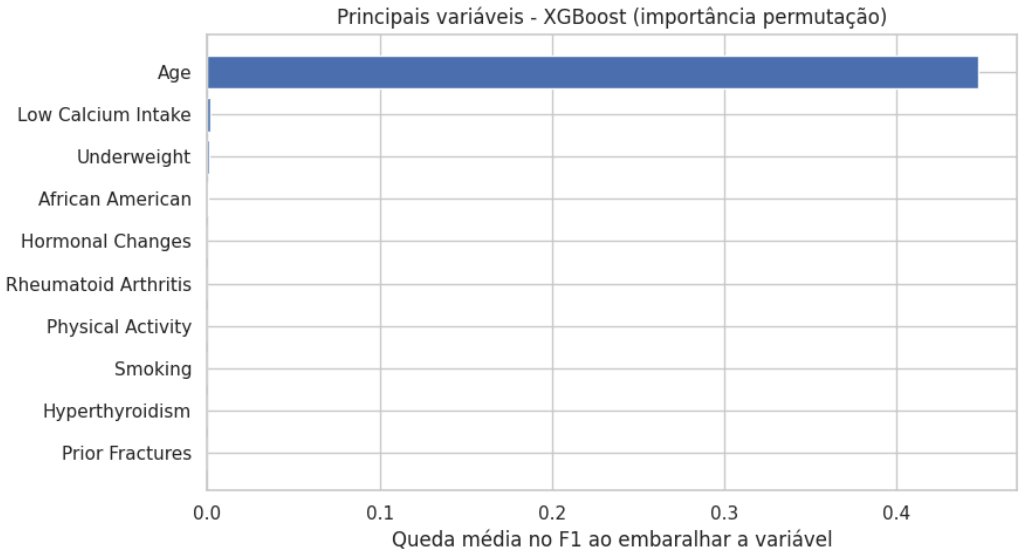


Imagem 14 - Gráfico relacionado à importância por permutação

5 RESULTADOS E ESCOLHA DO MELHOR MODELO

A comparação entre os modelos de Regressão Logística, Random Forest e Gradient Boosting (XGBoost) foi feita utilizando as métricas de acurácia, precisão, recall, F1-score e ROC-AUC, no conjunto de validação no conjunto de teste.

A Regressão Logística apresentou F1-score de 83,39% na validação e 77,52% no teste. O Random Forest obteve 90,30% e 85,16%, respectivamente, enquanto o XGBoost alcançou 89,96% e 84,82%. Dessa forma, o Random Forest apresentou o maior F1-score nas duas etapas, indicando o melhor equilíbrio entre precisão e recall segundo essa métrica.

Em relação à ROC-AUC, o XGBoost apresentou os melhores resultados, com 92,46% na validação e 87,98% no teste, ligeiramente superiores aos do Random Forest. Entretanto, este último também obteve a maior acurácia no teste, de 87,07%, além de precisão de 100%.

Os resultados demonstram que o Random Forest apresentou o melhor desempenho geral segundo os critérios considerados, embora o XGBoost tenha se destacado na capacidade de discriminação entre as classes. Observou-se também uma redução das métricas no conjunto de teste, reforçando a importância de avaliar a generalização dos modelos. No contexto da osteoporose, o recall merece atenção especial, pois representa a proporção de casos positivos corretamente identificados.

	Modelo	Etapa	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Regressão Logística	Validação	0.8469	0.9113	0.7687	0.8339	0.9133
1	Regressão Logística	Teste	0.8027	0.9009	0.6803	0.7752	0.8633
2	Regressão Logística	Variação (Val - Test)	0.0442	0.0104	0.0884	0.0588	0.0499
3	Random Forest	Validação	0.9116	1.0000	0.8231	0.9030	0.9232
4	Random Forest	Teste	0.8707	1.0000	0.7415	0.8516	0.8788
5	Random Forest	Variação (Val - Test)	0.0408	0.0000	0.0816	0.0514	0.0444
6	XGBoost	Validação	0.9082	0.9918	0.8231	0.8996	0.9246
7	XGBoost	Teste	0.8673	0.9909	0.7415	0.8482	0.8798
8	XGBoost	Variação (Val - Test)	0.0408	0.0009	0.0816	0.0514	0.0448

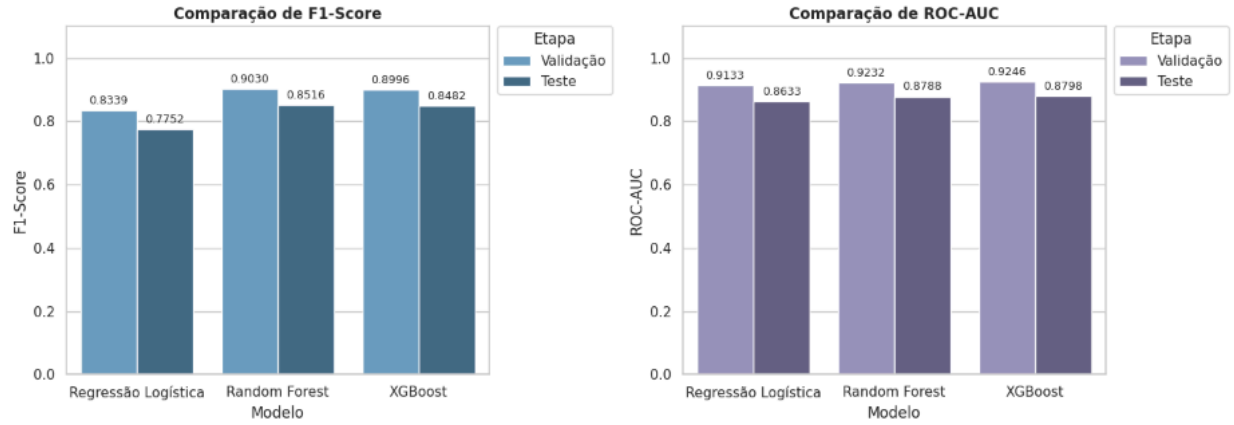


Imagem 15 – Comparação entre resultados de modelos

5.1 Modelo escolhido

O modelo Random Forest foi selecionado por apresentar os maiores valores de F1-score na validação (90,30%) e no teste (85,16%), além da maior acurácia no teste, de 87,07%. Sua precisão foi de 100% nas duas etapas avaliadas, indicando que todas as previsões positivas realizadas nesses conjuntos corresponderam a registros positivos.

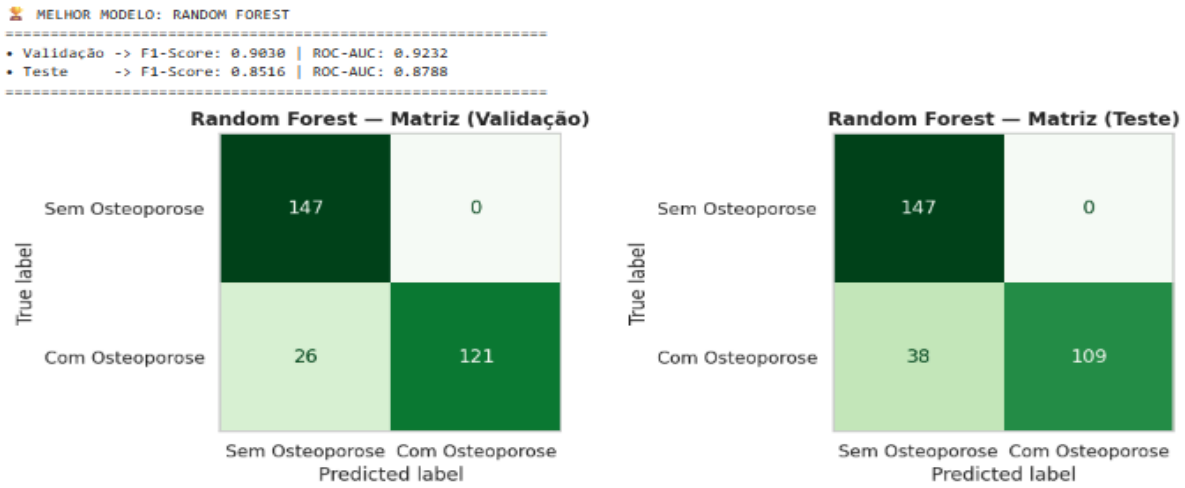


Imagem 16 – Modelo Escolhido

Apesar desses resultados, o recall do modelo foi de 74,15% no teste, demonstrando que parte dos registros com osteoporose não foi identificada.

Portanto, embora o desempenho seja promissor para a classificação experimental dos dados, o modelo apresenta limitações que precisam ser consideradas.

Assim, o Random Forest foi escolhido com base na comparação conjunta das métricas, especialmente pelo F1-score. Seu uso deve ser considerado auxiliar e experimental, não substituindo a avaliação clínica ou a decisão final do médico. Antes de uma aplicação prática, são necessárias avaliações adicionais com dados clínicos representativos.

6 DESAFIO

De acordo com o desafio adicional proposto, acrescentamos uma etapa de análise fundamentada em visão computacional, utilizando um conjunto de dados de exames de imagem direcionado ao diagnóstico da osteoporose. Com esse *dataset*, foram treinados modelos baseados em redes neurais convolucionais (CNNs) para a classificação das imagens de raio-X, permitindo prever se os pacientes apresentam osteoporose, osteopenia ou se estão saudáveis, enriquecendo o modelo preditivo e fortalecendo o suporte à tomada de decisão médica.

6.1 Base de dados

Assim como na etapa inicial da pesquisa, o conjunto de dados de exames de imagem (*Knee Osteoporosis Dataset multiclasses*) foi extraído da plataforma Kaggle[1]. A base é composta por exames de raio-X de joelho organizados em três diretórios distintos representando as classes do diagnóstico: a pasta normal, contendo 36 imagens; a pasta osteopenia, composta por 154 imagens; e a pasta osteoporosis, reunindo 49 imagens. Além das imagens, a base conta com um arquivo complementar contendo informações detalhadas dos pacientes.

[1] Fonte Disponível em: https://www.kaggle.com/datasets/mohamedgobara/osteoporosis-database.

6.2 Manipulação

Inicialmente foi feito uma varredura nas pastas procurando imagens com tres tipos de extensões para confirmar a quantidade de imagens de cada pasta , as extensões foram .png .jpg e .jpeg ,depois da validação foram encontradas as mesmas quantidades descritas no site dataset descrito pela Kaggle.

Devido a baixa quantidade de imagens , os treinos anteriores traziam baixa acurácia , a solução foram a criação de novas imagens a partir da base original,

padrões foram seguidos para não alterar as imagens a modo que interferissem no treinamento e trouxessem valores irreais, os seguintes parâmetros para criação das imagens foram :

```
datagen = ImageDataGenerator(
    rotation_range=15,       # Rotação suave de até 15 graus
    width_shift_range=0.05,  # Leve deslocamento horizontal
    height_shift_range=0.05, # Leve deslocamento vertical
    zoom_range=0.05,         # Leve zoom
    horizontal_flip=True,    # Inversão horizontal (útil se a simetria fizer sentido no exame)
    fill_mode='nearest'      # ( Preenche os pixels vazios gerados pelas transformações
                             # repetindo a cor do pixel da borda mais próximo.)
)
```

Imagem 17 - Padronização para criação de novas imagens

Em seguida foi criado um algoritmo para geração das novas imagens, funcionando como um assistente organizado que garante que todas as categorias de fotos do projeto tenham exatamente 500 imagens. O processo começa com o programa definindo essa meta e olhando, uma por uma e em ordem alfabética, cada pasta do dataset onde as imagens estão guardadas. Para cada pasta acessada, ele roda uma verificação rápida para contar quantas fotos válidas já existem ali dentro.

Em seguida, o programa analisa o total encontrado. Se a pasta já tiver 500 fotos ou mais, ele entende que a meta foi atingida, mostra uma mensagem de aviso e passa imediatamente para a próxima pasta. Se a pasta tiver menos do que 500 fotos, ele calcula exatamente quantas faltam criar para completar a meta. Para gerar essas novas imagens, o código prioriza o uso das fotos originais do projeto, separando-as daquelas que foram criadas em testes anteriores. Caso a pasta esteja totalmente vazia e sem nenhuma foto para usar como referência, ele avisa que não é possível continuar e pula aquela categoria por segurança.

Com as fotos originais selecionadas, o programa organiza uma fila onde essas imagens vão se revezando continuamente. Ele cria uma barra de progresso na tela para acompanhar o trabalho e divide a tarefa em pequenos blocos de até 100 fotos por vez. Para cada nova imagem necessária, o sistema pega uma foto da fila, ajusta o seu formato para que o computador consiga processá-la e aplicar alterações visuais pré estabelecidas, salvando a nova versão modificada diretamente na mesma pasta. Ao final de cada bloco de 100 fotos, o programa faz uma faxina na memória do computador para evitar travamentos, repetindo esse ciclo com uma checagem adicional, até a meta e printar que o trabalho foi concluído.



Imagem 18 - Saída do algoritmo de criação das imagens

6.3 Divisão treino, validação e teste

Para o treinamento foram estabelecidos três valores a serem seguidos, 70% dos dados ficariam para a base de treinamento para os modelos , 15% para a validação dos treinamentos e 15% dos dados para testes posteriores de acurácia dos modelos.

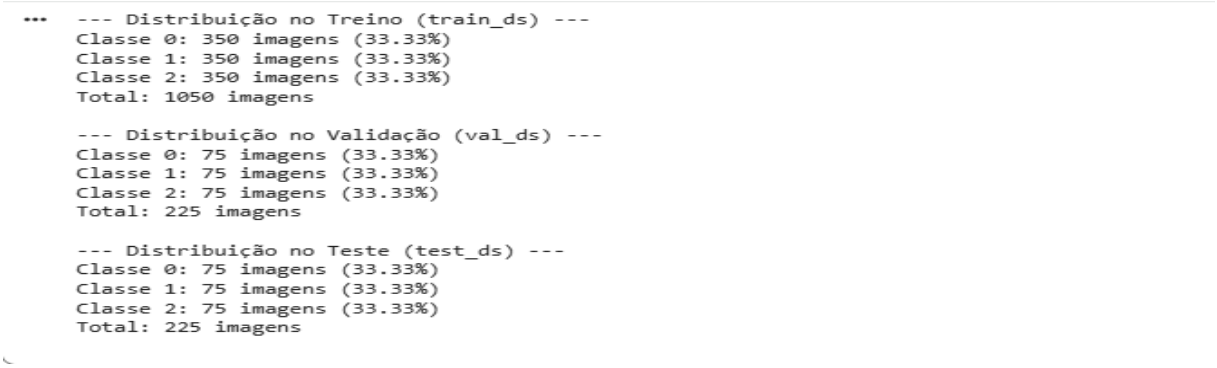


Imagem 19 - Distribuição entre Treino, Validação e Teste

O algoritmo começa com o código varrendo a pasta principal do dataset para identificar as subpastas que serão as categorias existentes e através delas é criado um índice numerado, cada pasta ganha um número de identificação exclusivo, no dataset em questão a classe normal ficou com índice 0, a classe osteopenia ficou com índice 1 e a classe osteoporosis com indice 2. Em seguida, ele percorre cada uma dessas pastas, encontra todas as imagens válidas e guarda o endereço exato de onde a foto está salva no computador junto com o número correspondente à sua categoria.

Com a lista completa de imagens e categorias em mãos, o programa realiza uma divisão do material em três grupos distintos, garantindo que a proporção de

cada categoria seja mantida em todas as etapas. Na primeira divisão, ele separa 70% de todo o acervo para formar o grupo de treino. Os 30% restantes são separados e, logo em seguida, divididos ao meio, resultando em 15% para o grupo de validação e 15% para o grupo de teste.

Por fim, o algoritmo estabelece uma rotina automatizada para o carregamento e o pré-processamento eficiente das imagens, preparando-as para o consumo pela inteligência artificial sem sobrecarregar a memória do sistema, cada arquivo é lido diretamente do disco no momento necessário, convertido para o formato de cores adequado e redimensionado para um padrão fixo de 224 por 224 pixels.

Em seguida, o código agrupa esses dados em lotes padronizados de 25 imagens por lote. No conjunto de treinamento, a ordem das imagens é embaralhada a cada ciclo para evitar que o modelo memorize sequências e desenvolva vício de aprendizado. Para otimizar o tempo de processamento, o sistema utiliza o mecanismo de pré-carregamento em segundo plano (*prefetch*), permitindo que o próximo lote de imagens seja preparado na memória enquanto o lote atual está sendo processado pelo modelo, garantindo o fluxo contínuo dos dados.

Em seguida é impresso o primeiro lote para analisar a qualidade das imagens a serem treinadas e se realmente o sistema está separando aleatoriamente as imagens.

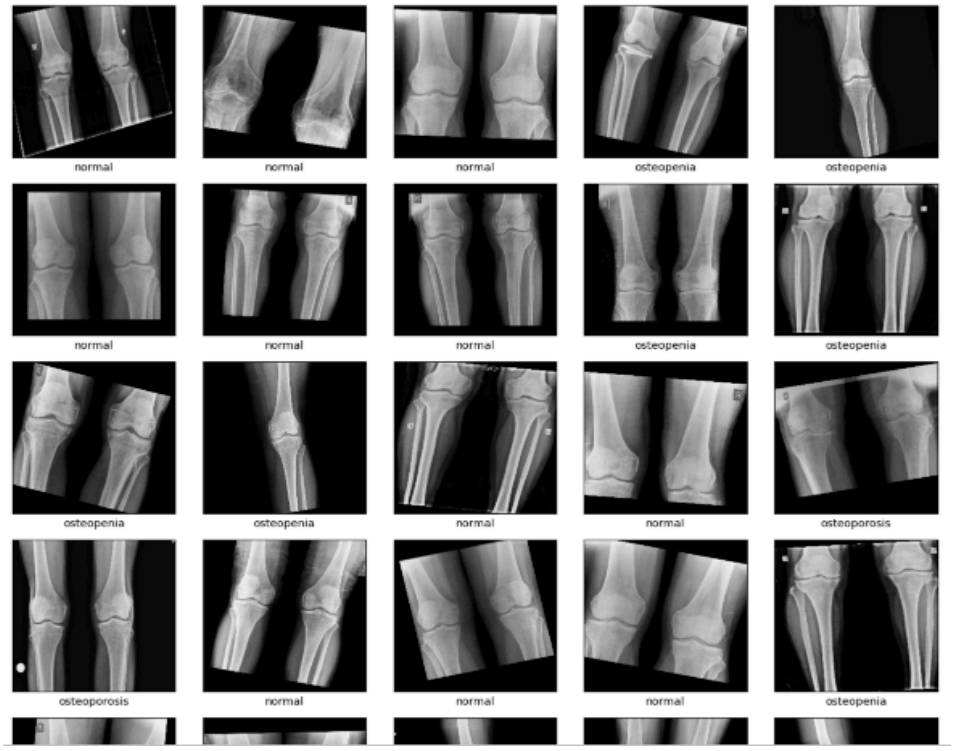


Imagem 20 - Impressão do Primeiro Lote de Treinamento

Um algoritmo foi criado para realizar um levantamento de todas as imagens do grupo de treinamento e calcular os pesos de cada categoria por meio de uma fórmula matemática. Por padrão, categorias com menos fotos recebem um peso maior, o que funciona como uma regra onde errar uma foto de uma classe rara custa mais caro para o sistema. Dessa forma, a inteligência artificial é induzida a dar a mesma atenção a todas as classes, sem priorizar aquelas com maior volume de dados.

Mesmo quando as classes possuem a mesma quantidade de imagens, dar pesos diferenciados às categorias pode ser fundamental para elevar a acurácia geral do modelo e evitar que ele decore os dados. Isso acontece porque algumas categorias apresentam detalhes visuais mais complexos do que outras. Sem um ajuste na penalização dos erros, o sistema tende a se acomodar nas classes mais fáceis e errar nas mais difíceis. Atribuir pesos maiores às classes de maior complexidade corrige essa distorção ao penalizar o modelo com mais rigor quando ele erra nesses casos. Somado a isso, o uso de rotações e zooms aleatórios garante que o sistema aprenda características genéricas em vez de memorizar imagens específicas.

6.4 Treinamentos

Após a realização de experimentos preliminares, estabeleceu-se uma padronização para o processo de treinamento. Cada modelo foi configurado para que em cada rodada de treinamento, descongelamos cerca de 26% das camadas do modelo[1], para que os modelos conseguissem adaptar suas representações visuais mais complexas às particularidades do nosso conjunto de dados, permitindo que a rede aprenda padrões específicos da nova tarefa sem perder ou 'destruir' os recursos visuais mais genéricos que ela já havia aprendido. Com isso, conseguimos elevar o desempenho da arquitetura, a técnica foi usada visando obter maior acurácia nos treinamentos.

Camadas do modelo[1] : consistem em blocos encadeados que processam os dados progressivamente, onde as camadas iniciais identificam padrões simples (como bordas e cores), as intermediárias agrupam esses elementos em formas e texturas, e as finais combinam tudo para reconhecer conceitos complexos e gerar a previsão na camada de saída.

```python
# Definir padrão aleatoriedade no início da iteração
np.random.seed(42)
tf.random.set_seed(42)

# Carregar Treinamento
treino_mobile = MobileNet(
    input_shape=(224, 224, 3),
    weights="imagenet",
    include_top=False,
)

# Congelar quase todas as camadas, exceto os ultimos 26% da rede
treino_mobile.trainable = True
for camada in treino_mobile.layers[:-40]:  # decongelar 26% da rede
  camada.trainable = False

model_mobilenet = tf.keras.models.Sequential(name="Modelo_MobileNetV1")
model_mobilenet.add(tf.keras.layers.Input(shape=(224, 224, 3)))
model_mobilenet.add(tf.keras.layers.Lambda(tf.keras.applications.mobilenet.preprocess_input))
model_mobilenet.add(treino_mobile)
model_mobilenet.add(tf.keras.layers.GlobalAveragePooling2D())
model_mobilenet.add(tf.keras.layers.Dropout(0.3))
model_mobilenet.add(tf.keras.layers.Dense(16, activation="relu"))
model_mobilenet.add(tf.keras.layers.Dropout(0.3))
model_mobilenet.add(tf.keras.layers.Dense(3, activation="softmax"))

model_mobilenet.compile(
  optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
  loss=tf.keras.losses.SparseCategoricalCrossentropy(),
  metrics=["accuracy"],
)
```

Imagem 21 - Exemplo Treinamento MobileNet

Todos os treinamentos mantiveram a mesma estrutura base, alterando-se apenas as nomenclaturas de identificação e a chamada da arquitetura pré-treinada. Na linha destacada em azul, substituiu-se para o modelo ResNet50 o parâmetro para applications.ResNet50, enquanto para o modelo EfficientNetB0 essa etapa de reescalonamento manual foi removida, uma vez que a própria biblioteca já realiza a normalização dos pixels[1] internamente. Os demais hiperparâmetros e camadas da estrutura foram mantidos idênticos para todos os treinamentos, conforme detalhado na Tabela 5

Normalização de pixels[1] : é a conversão dos pixels da imagem (originalmente entre 0 e 255) para uma faixa menor, como [-1, 1], necessária para o bom treinamento do modelo.

Tabela 5 – Parâmetros e configurações padronizadas adotadas no treinamento dos modelos.

Parâmetro / Configuração	Valor / Especificação	Função no Pipeline
Resolução de Entrada	$224 \times 224 \times 3$	A imagem de Raio-X é replicada nos 3 canais (RGB) para garantir compatibilidade com o formato de entrada padrão das redes neurais pré-treinadas.
Camada de Agregação	GlobalAveragePooling2D	GlobalAveragePooling2D é uma camada de CNNs que resume cada mapa de características na sua média, transformando dados 2D num único vetor para deixar o modelo mais leve e eficiente.

Parâmetro / Configuração	Valor / Especificação	Função no Pipeline
Primeira Regularização	Dropout (Taxa = 30%)	Camada que desativa 30% da rede ao acaso durante o treino para combater o decorismo e tornar o aprendizado mais genérico e robusto.
Camada Densa Oculta	16 neurônios (ReLU)	É uma camada densa que combina as informações aprendidas em 16 padrões diferentes, usando a função ReLU para filtrar ruídos negativos e aprender relações complexas.
Segunda Regularização	Dropout (Taxa = 30%)	Camada que desativa mais 30% da rede ao acaso durante o treino para combater o decorismo e tornar o aprendizado mais genérico e robusto.
Camada de Saída	3 neurônios (Softmax)	É a camada final que devolve a probabilidade da imagem pertencer a cada uma das 3 classes, permitindo escolher a opção com maior percentagem de certeza.

6.4.1 MobileNet

O modelo MobileNet foi treinado inicialmente assim como todos os outros modelos ao longo de 50 épocas. Porém, após testes e refinamento da criação padrão dos treinamentos de todos os modelos, observou-se que a acurácia de validação (val_accuracy) começa a estagnar e sofrer overfitting depois de 20 épocas. Embora a acurácia máxima pontual tenha sido atingida na época 49 (0,8667), a definição de um limite em 20 épocas mostra-se o parâmetro mais adequado para evitar o sobreajuste e garantir um treino eficiente."

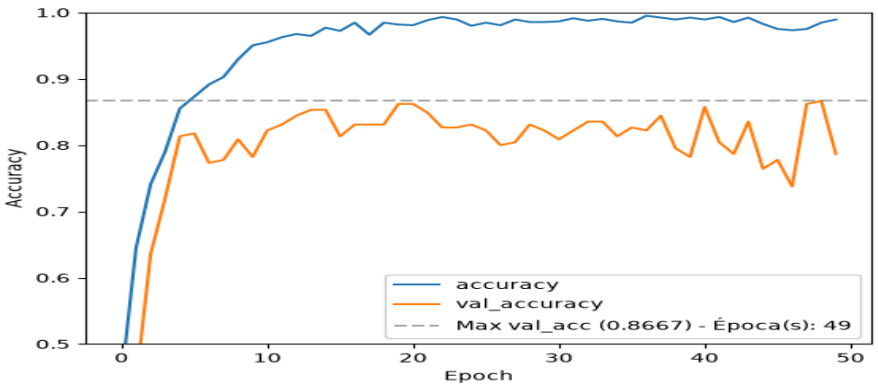


Imagem 22 - Treino MobileNet 50 épocas

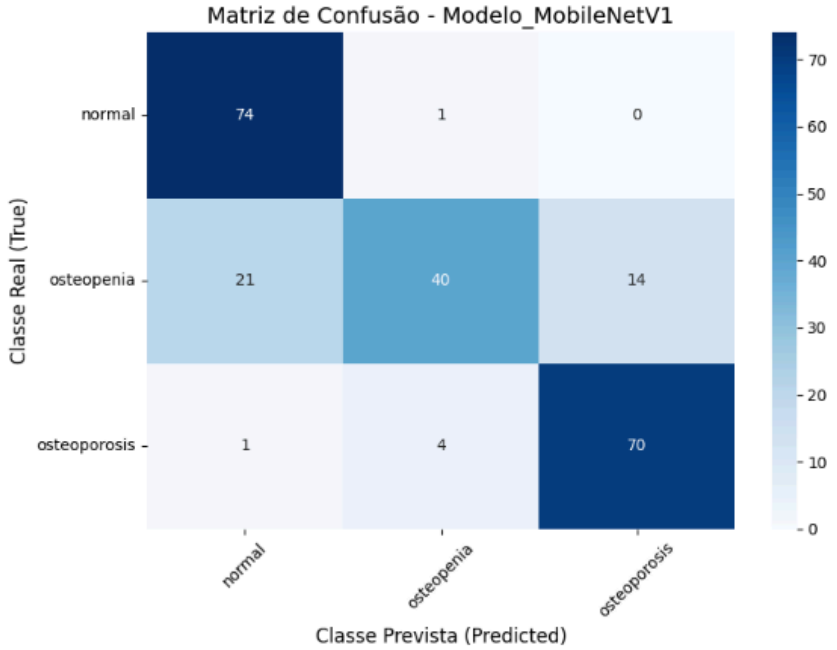
Após cumprir a rotina de 20 épocas, o modelo alcançou uma acurácia de 81,78% no conjunto de teste, representando o maior valor atingido entre todos os testes realizados.

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet/mobilenet_1_0_224_tf_no_top.h5
17225924/17225924 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Epoch 1/20
42/42 ━━━━━━━━━━━━━━━━━━━━ 228s 5s/step - accuracy: 0.4352 - loss: 1.1658 - val_accuracy: 0.3911 - val_loss: 1.1653
Epoch 2/20
42/42 ━━━━━━━━━━━━━━━━━━━━ 219s 5s/step - accuracy: 0.6257 - loss: 0.8224 - val_accuracy: 0.4667 - val_loss: 1.0324
Epoch 3/20
42/42 ━━━━━━━━━━━━━━━━━━━━ 261s 5s/step - accuracy: 0.6629 - loss: 0.6944 - val_accuracy: 0.5511 - val_loss: 0.8390
Epoch 4/20
42/42 ━━━━━━━━━━━━━━━━━━━━ 193s 5s/step - accuracy: 0.7505 - loss: 0.5642 - val_accuracy: 0.6489 - val_loss: 0.7657
Epoch 5/20
42/42 ━━━━━━━━━━━━━━━━━━━━ 224s 5s/step - accuracy: 0.8067 - loss: 0.4395 - val_accuracy: 0.7867 - val_loss: 0.5584
--------------------------------------------------------------------------------------------------------------------------
42/42 ━━━━━━━━━━━━━━━━━━━━ 210s 5s/step - accuracy: 0.9829 - loss: 0.0674 - val_accuracy: 0.9111 - val_loss: 0.4289
Epoch 19/20
42/42 ━━━━━━━━━━━━━━━━━━━━ 200s 5s/step - accuracy: 0.9838 - loss: 0.0607 - val_accuracy: 0.8756 - val_loss: 0.5801
Epoch 20/20
42/42 ━━━━━━━━━━━━━━━━━━━━ 209s 5s/step - accuracy: 0.9857 - loss: 0.0519 - val_accuracy: 0.8756 - val_loss: 0.6096

--- Avaliando no conjunto de teste ---
9/9 ━━━━━━━━━━━━━━━━━━━━ 30s 3s/step - accuracy: 0.8178 - loss: 0.7557
Acurácia de Teste: 81.78%
```

Imagem 23 - Treino MobileNet 20 épocas

A análise da matriz de confusão e das métricas de desempenho do modelo demonstrou elevada capacidade de identificação para exames normais, atingindo um recall de 98,67%. A classe osteoporosis apresentou bom desempenho, com precisão de 83,33%, recall de 93,33% e f1-score de 88,05%. Em contrapartida, a maior taxa de erro concentrou-se na classe osteopenia, que obteve recall de 53,33%; seus casos foram predominantemente confundidos com as classes normal 21 casos e osteoporosis 14 casos, devido ao seu caráter intermediário entre a normalidade e a patologia severa.




```
--- Relatório de Classificação: Modelo_MobileNetV1 ---
              precision    recall  f1-score   support

      normal     0.7708    0.9867    0.8655        75
  osteopenia     0.8889    0.5333    0.6667        75
osteoporosis     0.8333    0.9333    0.8805        75

    accuracy                         0.8178       225
   macro avg     0.8310    0.8178    0.8042       225
weighted avg     0.8310    0.8178    0.8042       225
```

Imagem 24 - Matriz de Confusão MobileNet 20 épocas

6.4.2 ResNet50

O modelo ResNet foi treinado inicialmente, assim como todos os outros modelos, ao longo de 50 épocas. Porém, durante o processo de treinamento, observou-se que a acurácia de validação (val_accuracy) atingiu seu melhor patamar na época 34, apresentando instabilidade e sinais severos de overfitting nas épocas seguintes, a definição de um limite em 34 épocas mostra-se o parâmetro mais adequado para preservar a generalização e garantir um treino eficiente.

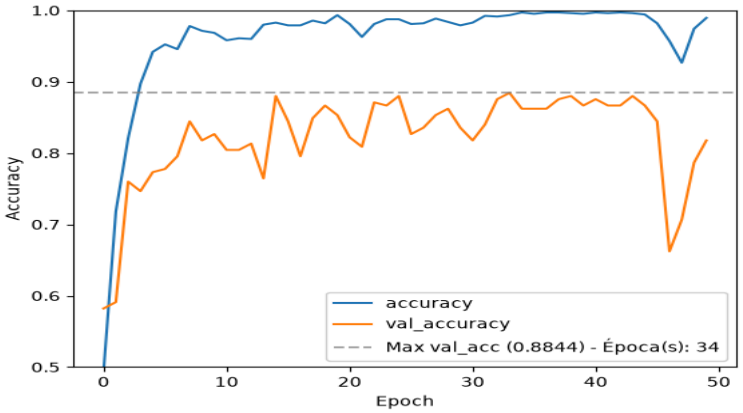


Imagem 25 - Treino RestNet50 50 épocas

Após cumprir a rotina de 34 épocas, o modelo alcançou uma acurácia de 80,89% no conjunto de teste, representando o maior valor atingido para esse modelo entre todos os testes realizados.

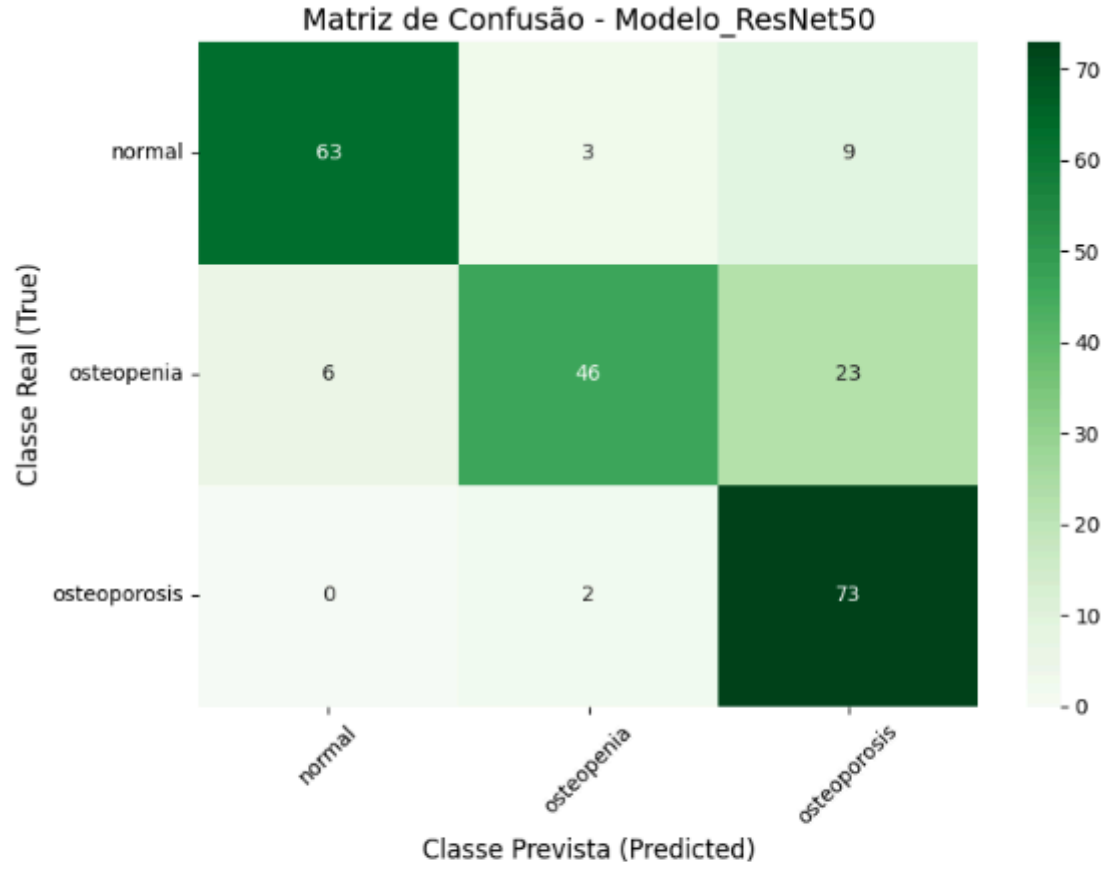
```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/34
42/42 ━━━━━━━━━━━━━━━━━━━━ 502s 11s/step - accuracy: 0.4914 - loss: 1.0558 - val_accuracy: 0.5556 - val_loss: 0.9080
Epoch 2/34
42/42 ━━━━━━━━━━━━━━━━━━━━ 466s 11s/step - accuracy: 0.7162 - loss: 0.6464 - val_accuracy: 0.6844 - val_loss: 0.7716
Epoch 3/34
42/42 ━━━━━━━━━━━━━━━━━━━━ 479s 11s/step - accuracy: 0.8581 - loss: 0.3549 - val_accuracy: 0.7911 - val_loss: 0.7606
Epoch 4/34
42/42 ━━━━━━━━━━━━━━━━━━━━ 488s 11s/step - accuracy: 0.9086 - loss: 0.2293 - val_accuracy: 0.8222 - val_loss: 0.6220
Epoch 5/34
42/42 ━━━━━━━━━━━━━━━━━━━━ 481s 11s/step - accuracy: 0.9467 - loss: 0.1504 - val_accuracy: 0.8089 - val_loss: 1.3328
...
Epoch 32/34
42/42 ━━━━━━━━━━━━━━━━━━━━ 530s 13s/step - accuracy: 0.9876 - loss: 0.0324 - val_accuracy: 0.8267 - val_loss: 0.7637
Epoch 33/34
42/42 ━━━━━━━━━━━━━━━━━━━━ 473s 11s/step - accuracy: 0.9857 - loss: 0.0536 - val_accuracy: 0.8000 - val_loss: 0.8941
Epoch 34/34
42/42 ━━━━━━━━━━━━━━━━━━━━ 431s 10s/step - accuracy: 0.9762 - loss: 0.1142 - val_accuracy: 0.8133 - val_loss: 1.1671

--- Avaliando no conjunto de teste ---
9/9 ━━━━━━━━━━━━━━━━━━━━ 54s 5s/step - accuracy: 0.8089 - loss: 1.0740
Acurácia de Teste: 80.89%
```

Imagem 26 - Treino RestNet50 34 épocas

O modelo ResNet50 destacou-se pela excelente capacidade de identificação da condição severa (osteoporosis), atingindo recall de 97,33% 73 de 75 exames identificados corretamente. Os exames da classe normal mantiveram bom desempenho, com precisão de 91,30% e recall de 84,00%. Por outro lado, a classe osteopenia apresentou a menor sensibilidade recall de 61,33%. Dos 75 exames de osteopenia, os casos foram predominantemente confundidos com a classe

osteoporosis 23 casos e, em menor escala, com a classe normal 6 casos, devido ao seu caráter intermediário entre a normalidade e a patologia severa.




```
--- Relatório de Classificação: Modelo_ResNet50 ---
              precision    recall  f1-score   support

      normal     0.9130    0.8400    0.8750        75
  osteopenia     0.9020    0.6133    0.7302        75
osteoporosis     0.6952    0.9733    0.8111        75

    accuracy                         0.8089       225
   macro avg     0.8367    0.8089    0.8054       225
weighted avg     0.8367    0.8089    0.8054       225
```

Imagem 27 - Matriz de Confusão RestNet50 34 épocas

6.4.3 EfficientNetB0

No caso do treinamento com o modelo EfficientNetB0, o treino inicial também correu ao longo de 50 épocas. Observou-se , que a rede aprende muito depressa e estabiliza o seu desempenho por volta da época 30. A acurácia de validação (val_accuracy) atinge o seu pico máximo de 0,9378 logo na época 36. Embora este mesmo topo volte a surgir nas épocas 39 e 40, em seguida o modelo começa a dar pequenos sinais de queda com a validação a oscilar ligeiramente para baixo enquanto o treino continua muito alto. Por isso, optou-se por interromper o treino na

época 36, garantimos o ponto mais alto de precisão, economizamos tempo de processamento e evitamos que a rede comece a memorizar os dados em demasia.

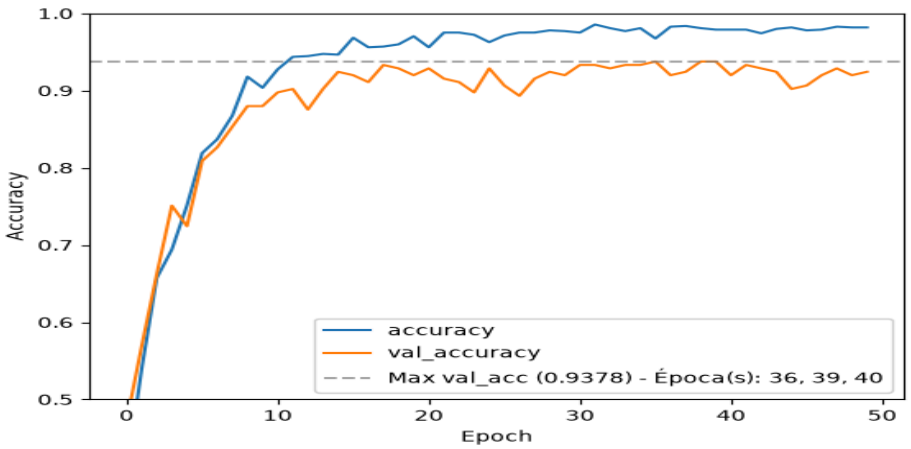


Imagem 28 - Treino EfficientNetB0 50 épocas

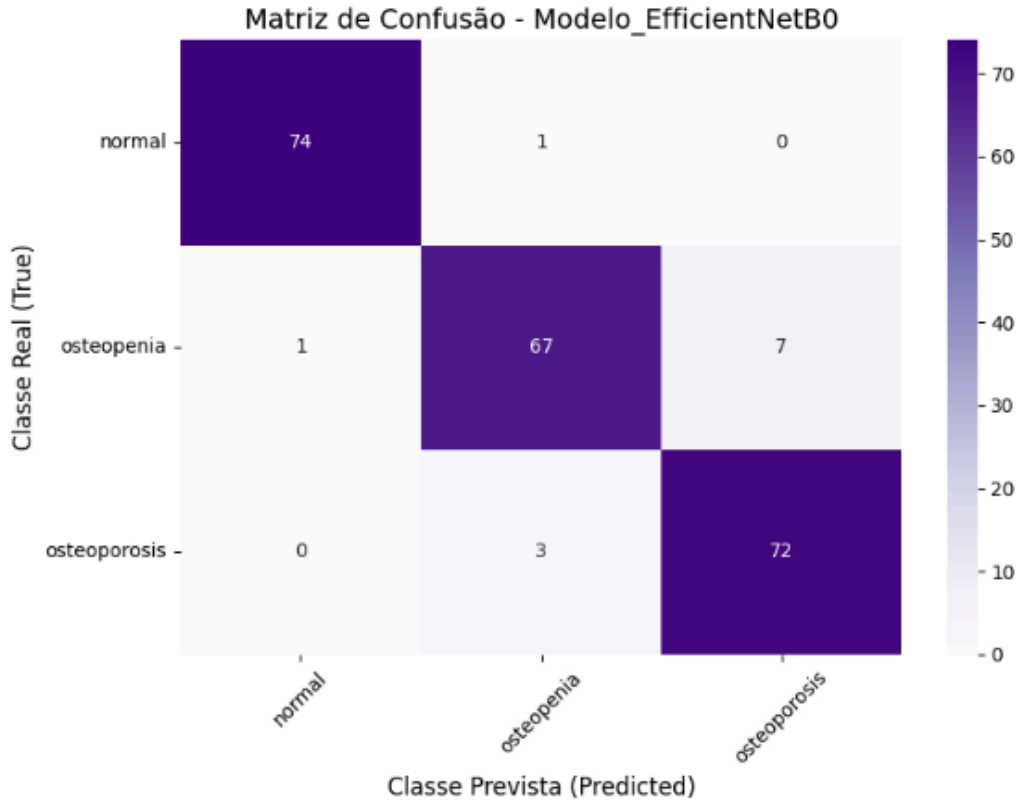
Após cumprir a rotina de 36 épocas, o modelo alcançou uma acurácia de 94,67% no conjunto de teste, representando o maior valor atingido para esse modelo entre todos os testes realizados.

```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop.h5
16705208/16705208 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/36
42/42 ━━━━━━━━━━━━━━━━━━━━ 238s 5s/step - accuracy: 0.3914 - loss: 1.0903 - val_accuracy: 0.4267 - val_loss: 1.0429
Epoch 2/36
42/42 ━━━━━━━━━━━━━━━━━━━━ 207s 5s/step - accuracy: 0.5543 - loss: 0.9392 - val_accuracy: 0.6311 - val_loss: 0.8919
Epoch 3/36
42/42 ━━━━━━━━━━━━━━━━━━━━ 200s 5s/step - accuracy: 0.6600 - loss: 0.8015 - val_accuracy: 0.7289 - val_loss: 0.7206
Epoch 4/36
42/42 ━━━━━━━━━━━━━━━━━━━━ 224s 5s/step - accuracy: 0.7362 - loss: 0.6790 - val_accuracy: 0.7956 - val_loss: 0.5816
Epoch 5/36
42/42 ━━━━━━━━━━━━━━━━━━━━ 217s 5s/step - accuracy: 0.7867 - loss: 0.5704 - val_accuracy: 0.8711 - val_loss: 0.4571
--------------------------------------------------------------------------------------------------------------------------------
42/42 ━━━━━━━━━━━━━━━━━━━━ 202s 5s/step - accuracy: 0.9800 - loss: 0.0582 - val_accuracy: 0.9333 - val_loss: 0.2254
Epoch 34/36
42/42 ━━━━━━━━━━━━━━━━━━━━ 207s 5s/step - accuracy: 0.9714 - loss: 0.0760 - val_accuracy: 0.9467 - val_loss: 0.2800
Epoch 35/36
42/42 ━━━━━━━━━━━━━━━━━━━━ 202s 5s/step - accuracy: 0.9771 - loss: 0.0639 - val_accuracy: 0.9422 - val_loss: 0.2747
Epoch 36/36
42/42 ━━━━━━━━━━━━━━━━━━━━ 213s 5s/step - accuracy: 0.9829 - loss: 0.0629 - val_accuracy: 0.9422 - val_loss: 0.2527
9/9 ━━━━━━━━━━━━━━━━━━━━ 30s 3s/step - accuracy: 0.9467 - loss: 0.2798
Acurácia de Teste EfficientNetB0: 94.67%
```

Imagem 29 - Treino EfficientNetB0 36 épocas

O modelo EfficientNetB0 demonstra um desempenho elevado, entre as três classes analisadas, a categoria normal destacou-se com os melhores resultados, alcançando 98,67% em todas as métricas principais precisão, *recall* e *f1-score*, com 74 de 75 exames corretamente identificados e apenas 1 caso incorretamente atribuído à osteopenia. A condição severa, osteoporosis, também obteve uma excelente taxa de identificação, com 96,00% de *recall* 72 acertos em 75 exames, 91,14% de precisão e 93,51% de *f1-score*, sendo os seus únicos 3 erros classificados como osteopenia. Por outro lado, a classe osteopenia, devido à sua natureza de transição entre a normalidade e a patologia grave, apresentou a menor

sensibilidade, com um *recall* de 89,33%, 67 acertos em 75, embora tenha mantido uma precisão alta de 94,37% e um *f1-score* de 91,78%. Dos 8 erros associados a esta classe real, 7 foram confundidos com osteoporose e 1 com a classe normal.




```
--- Relatório de Classificação: Modelo_EfficientNetB0 ---
              precision    recall  f1-score   support

      normal     0.9867    0.9867    0.9867        75
  osteopenia     0.9437    0.8933    0.9178        75
osteoporosis     0.9114    0.9600    0.9351        75

    accuracy                         0.9467       225
   macro avg     0.9472    0.9467    0.9465       225
weighted avg     0.9472    0.9467    0.9465       225
```

Imagem 30 - Matriz de Confusão EfficientNetB0 36 épocas

6.5 Comparando resultados

A avaliação comparativa do desempenho das redes neurais MobileNet, ResNet50 e EfficientNetB0 traz dados importantes sobre o comportamento e a estabilidade de cada arquitetura ao longo da fase de validação. Nos primeiros ciclos de treinamento, todas as redes iniciam uma trajetória ascendente de acurácia, partindo de patamares distintos em que a ResNet50 começa em um nível ligeiramente superior às demais. Contudo, à medida que o treinamento avança, a EfficientNetB0 rapidamente eleva sua acurácia para uma taxa elevada e mantém

uma linha contínua de estabilidade até a época 35. Por sua vez, a ResNet 50 exibe uma oscilação constante em sua acurácia, enquanto a MobileNet atinge um nível elevado em torno da época 17.

A análise complementar das curvas de perda reflete melhor a estabilidade do aprendizado de cada modelo e os riscos de sobreajuste. A EfficientNetB0 apresenta uma redução constante do erro desde as épocas iniciais, mantendo a menor taxa de perda durante todo o período sem flutuações expressivas. A MobileNet também demonstra uma trajetória descendente na perda durante as épocas em que esteve em execução, começando uma ascensão na perda a partir da época 15. Em contrapartida, a ResNet 50 exibe grande volatilidade em sua curva de erro, acompanhada por uma tendência de crescimento contínuo da perda nas épocas finais do treinamento, padrão que caracteriza o comportamento de sobreajuste.

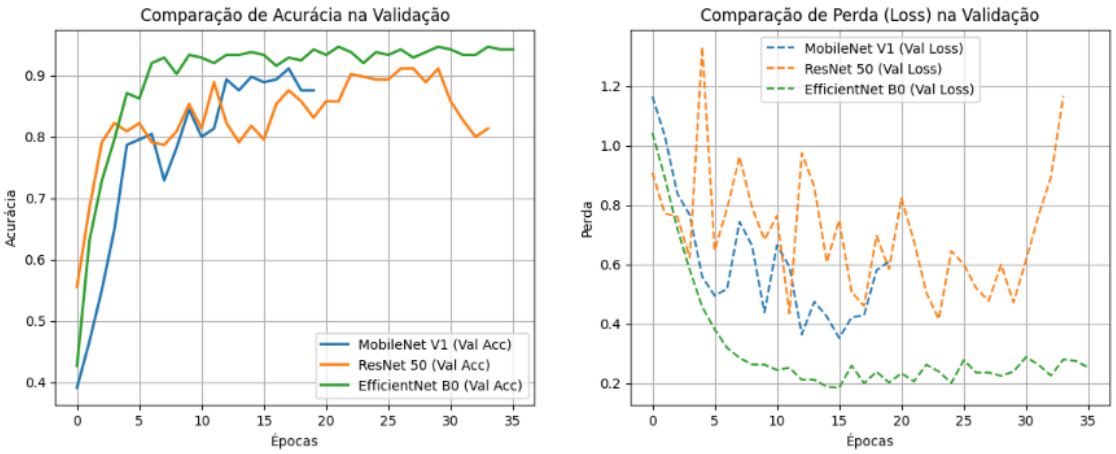


Imagem 31 - Comparação entre Modelos desafio

6.6 Modelo escolhido

A tabela comparativa final consolida os resultados quantitativos obtidos pelas três arquiteturas na fase de teste, avaliando a acurácia e a perda (*loss*) de cada modelo. A MobileNet atinge uma acurácia de teste de 81,78% acompanhada por uma perda de 0,7557%. A ResNet50 regista o menor valor de acurácia entre as três, com 80,89%, e a taxa de perda mais elevada do conjunto, situando-se nos 1,0740. Por sua vez, o EfficientNetB0 destaca-se ao alcançar uma acurácia de teste de 94,67% e ao registar a menor taxa de erro do grupo, com uma perda de 0,2798%. O resumo final da avaliação destaca a EfficientNetB0 como a arquitetura com o desempenho global mais sólido entre as opções analisadas no conjunto de modelos treinados .

```
=== TABELA COMPARATIVA ===
         Modelo Acurácia Teste (%) Perda (Loss) Teste
   MobileNet V1           81.7778%            0.7557
      ResNet 50           80.8889%            1.0740
EfficientNet B0           94.6667%            0.2798

=
🏆 MELHOR MODELO: EfficientNet B0
Acurácia: 94.6667% | Perda (Loss): 0.2798
=
```

Imagem 32 - Escolha do melhor modelo desafio

7 PERGUNTA FINAL

Discuta os resultados de maneira crítica. O seu modelo pode ser utilizado na prática? Como? (Lembre-se que o médico sempre deve ter a palavra final no diagnóstico).

Os resultados obtidos pelos treinamentos tanto por meio dos dados classificatórios dos pacientes tanto por análise de imagens de diferentes estágios de estruturas ósseas, dos mais variados pacientes demonstram que todo o trabalho desenvolvido apresenta aplicabilidade prática, podendo ser integrada à rotina clínica como um sistema de suporte à decisão médica. O diferencial do trabalho reside no uso auxiliar entre o aprendizado de máquina (Machine Learning) e o aprendizado profundo (Deep Learning).

Em um primeiro momento, o modelo faria a análise do perfil do paciente, um simples questionário composto por perguntas sobre dados clínicos, demográficos e comportamentais. como idade, gênero, hábitos de vida e dieta como consumo de cálcio, vitamina D, atividade física, tabagismo e álcool, histórico clínico e comorbidades como histórico familiar, uso de corticoides, fraturas prévias, artrite reumatóide e hipertireoidismo já ajudariam ao modelo fazer uma predição prévia da possibilidade, ao identificar uma probabilidade aumentada para o desenvolvimento da patologia, mesmo em fases assintomáticas, o algoritmo pode sugerir ao profissional de saúde a solicitação de exames radiológicos preliminares para aquele indivíduo. Essa abordagem favorece a triagem precoce na atenção primária, permitindo mapear pacientes em risco antes da progressão da perda óssea.

Em uma etapa posterior, durante a avaliação do exame radiográfico, o modelo de Deep Learning atuaria como uma ferramenta de visão computacional, tendo alcançado uma acurácia de 94,67% na diferenciação entre quadros normais, de osteopenia e de osteoporose mostrando-se uma ótima ferramenta em auxílio às tomadas de decisões médicas. Ao apontar automaticamente imagens com alto grau de suspeição, o modelo serve como um "segundo leitor" para o médico, assim

podendo encaminhar o paciente para exames confirmatórios de maior precisão, como a Densitometria Óssea (DXA) ou na indicação do plano terapêutico adequado.

Deixando bem claro que os estudos procuraram desenvolver treinamentos que podem ser usados para auxiliar os diagnósticos, porém palavra final e o diagnóstico definitivo cabe estritamente ao médico, que utilizará as sinalizações geradas pelos modelos para enriquecer sua avaliação clínica e garantir uma conduta individualizada e segura ao paciente.

7.1 Informações Complementares

Nesta seção, são apresentados os recursos adicionais, a composição da equipe responsável pela execução do projeto e os materiais audiovisuais de demonstração.

7.1.1 Repositório

O código-fonte completo, a documentação técnica e os arquivos de configuração desenvolvidos ao longo do projeto estão armazenados e mantidos em repositório público na plataforma GitHub. O acesso ao material pode ser feito pelo link: https://github.com/mariatozi/tech-challenge-fiap

7.1.2 Equipe de Desenvolvimento:

O planejamento, a arquitetura e a implementação da solução foram realizados de forma colaborativa pelos integrantes descritos a seguir:

Gabriel Lopes Guimarães - RM378344- https://github.com/Gabriel-lgui
Lucas Muniz Vieira Souto - RM378518 - https://github.com/Zenny-152
Marcos Pinheiro Moura - RM377337 - https://github.com/newcocas
Maria Luiza Assumpção Tozi - RM378682 - https://github.com/mariatozi
Saulo Roberto de Sena - RM378146 - https://github.com/senasaulo

7.1.3 Apresentação

Para demonstrar o funcionamento prático da aplicação e sintetizar as decisões de arquitetura adotadas, foi gravado um vídeo explicativo sobre o projeto, o acesso pode ser feito pelo link: https://www.youtube.com/watch?v=1KdVx5FaHKU

8 REFERÊNCIAS

PEDRO, Adriana Orcesi; PLAPLER, Pérola Grinberg; SZEJNFELD, Vera Lúcia (org.). Manual Brasileiro de Osteoporose: orientações práticas para os profissionais de saúde. 1. ed. São Paulo: Editora Clannad, 2021. Disponível em: https://osteoprevine.org.br/wp-content/uploads/2025/10/Manual-Brasileiro-de-Osteoporose.pdf. Acesso em: 16 set. 2026